



华南理工大学
South China University of Technology

操作系统实验报告

郑重声明：

- 1、实验手册中的所有实验均有本人独立编码、调试和测试。
- 2、实验手册中给出的实验数据和结果完全由本人所完成的程序给出。
- 3、本人了解：不按照前两条要求所完成的实验报告已经构成了抄袭或造假行为，本人将承担相应的不良后果。

姓名：_____（签名）

学号：_____

班级：_____22 计算机科学与技术 1 班

提交日期：_____2025/06/08

成绩：_____

一、实验目的

理解并行计算思想，通过线程将大规模向量点积任务分解，提高计算效率。

二、实验内容

在 Linux 平台上用多线程方法实现浮点向量的点积计算： $s = \sum_{i=0}^N a_i b_i$ 。

向量{a_i}和{b_i}的元素满足以下规则：
$$a_i = b_i = \begin{cases} 1.0 & (i \bmod 3 = 0) \\ -1.0 & (i \bmod 3 = 1) \\ 0 & (i \bmod 3 = 2) \end{cases}$$

三、实验环境

CPU			Cache			操作系统	编译器	
型号	核数	主频	L1	L2	L3		版本	优化参数
AMD EPYC 7C13 64-Core Processor	6	1996.249 Mhz	384 KiB	3MiB	1.5GiB	Debian 12.2.0	gcc version 12.2.0 (Debian 12.2.0-14+deb12u1)	-O2 -pthread

四、实验步骤（附上截图）

1. 固定 N，测试不同线程数 thread_num，记录执行时间。

```
root@erstidyfygiuhio:~# ./sy3 1 100000
s=66667.00 t=0.294021(ms)
root@erstidyfygiuhio:~# ./sy3 1 1000000
s=666667.00 t=1.658511(ms)
root@erstidyfygiuhio:~#
root@erstidyfygiuhio:~# ./sy3 1 10000000
s=6666667.00 t=19.670212(ms)
root@erstidyfygiuhio:~# ./sy3 1 100000000
s=66666667.00 t=108.835190(ms)
root@erstidyfygiuhio:~# ./sy3 1 1000000000
s=666666667.00 t=1030.505313(ms)
root@erstidyfygiuhio:~# ./sy3 1 10000000000
s=6666666667.00 t=10030.980151(ms)
```

2. 固定 线程数 thread_num = 1，测试不同规模的 N，记录执行时间。

```
root@erstidyfygiuhio:~# ./sy3 2 100000000
s=66666667.00 t=94.992178(ms)
root@erstidyfygiuhio:~# ./sy3 4 100000000
s=66666667.00 t=47.819395(ms)
root@erstidyfygiuhio:~# ./sy3 8 100000000
s=66666667.00 t=23.829793(ms)
root@erstidyfygiuhio:~# ./sy3 16 100000000
s=66666667.00 t=25.582350(ms)
```

五、实验结果及分析

计算结果和计算时间

thread_num	N	计算结果	计算时间 (ms)
1	100000	66667.00	0.294021
1	1000000	666667.00	1.658511
1	10000000	6666667.00	19.670212
1	100000000	66666667.00	108.835190
2	100000000	66666667.00	94.992178
4	100000000	66666667.00	47.819395
8	100000000	66666667.00	23.829793
16	100000000	66666667.00	25.582350

图 1 的 X 轴为 N，Y 轴为 Thread_num 为 1 时的计算时间，

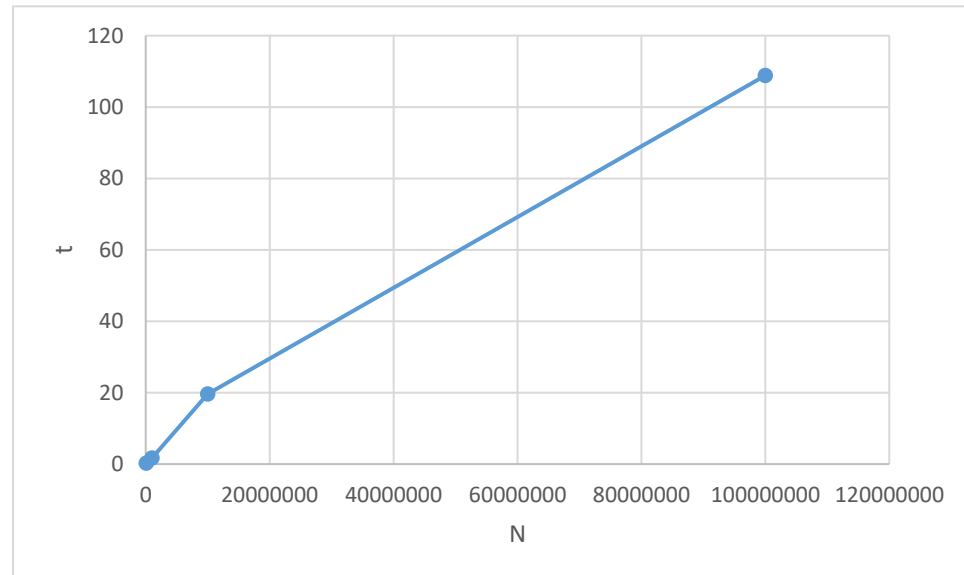
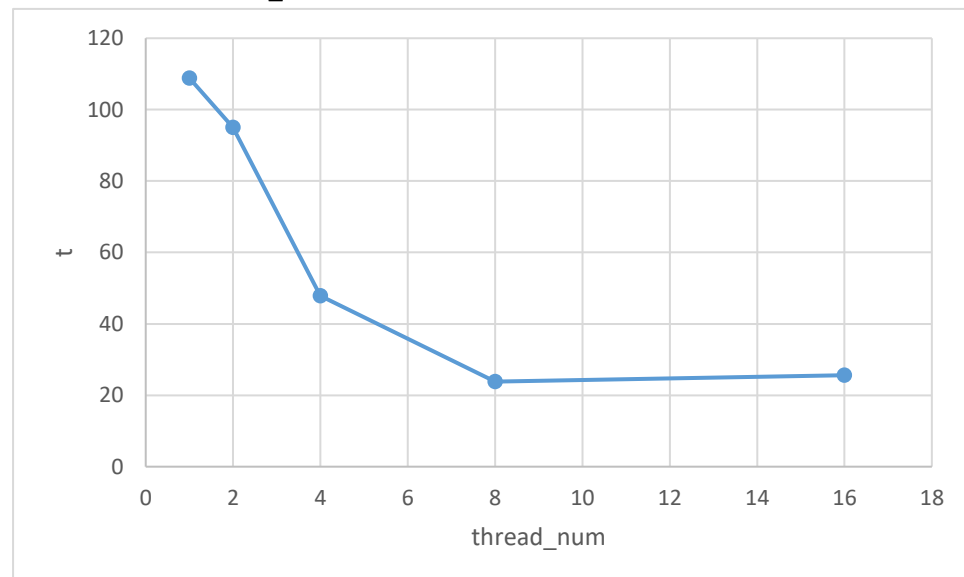


图 2 的 X 轴为 Thread_num，Y 轴为 N 为 100,000,000 时的计算时间。



如果你的 CPU 为多核处理器，你是否观察到计算加速了？

如果观察到计算加速的话，你的处理器核数是 6，图 2 中 Thread_num 为 8 时，计算性能最好，其加速比是 0.22。（加速比=多线程执行时间/单线程执行时间）

代码:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <time.h>

typedef struct {
    // 线程函数参数结构体
    // 用于传递给每个线程的计算参数和存储结果
    int thread_id; // 线程编号，[0, 总线程数-1]
    int thread_num; // 总线程数
    long N; // 向量长度
    double partial_sum; // 本线程计算得到的部分和
} thread_arg_t;

void *thread_func(void *arg) {
    // 线程执行函数
    // 计算分配给本线程的向量区间的点积

    thread_arg_t *targ = (thread_arg_t *)arg; // 将参数转换为期望类型
    long id = targ->thread_id; // 本线程的编号
    long num = targ->thread_num; // 总线程数
    long N = targ->N; // 向量总长度

    long chunk = N / num; // 计算每个线程处理的元素数目

    long start = id * chunk; // 计算本线程处理的起始索引

    long end = (id == num - 1) ? N : start + chunk; // 如为最后一个线程，则处理剩余所有元素；否则处理固定长度

    double sum = 0.0; // 局部点积和初始化
    for (long i = start; i < end; ++i) { // 遍历分配给本线程的元素区间
        double ai, bi;
        switch (i % 3) { // 根据 i 对 3 取余决定向量元素的值
            case 0: ai = bi = 1.0; break;
```

```

        case 1: ai = bi = -1.0; break;
        default: ai = bi = 0.0; break;
    }
    sum += ai * bi; //累加本次计算结果
}
targ->partial_sum = sum; //将本线程计算结果写回参数结构体
return NULL; //线程结束
}

int main(int argc, char *argv[]) {
    if (argc != 3) { // 检查命令行参数是否正确
        fprintf(stderr, "Usage: %s <thread_num> <N>\n", argv[0]);
        return EXIT_FAILURE;
    }

    int thread_num = atoi(argv[1]); // 将第一个参数转换为线程数量
    long N = atol(argv[2]); //将第二个参数转换为向量长度
    if (thread_num < 1 || N < 100000 || thread_num > 16) { //检查命令行参数是否正确
        fprintf(stderr, "参数错误: thread_num 需为 1~16 之间, N 需为不小于 100000\n");
        return EXIT_FAILURE;
    }
    // 动态分配存储线程句柄和参数结构体数组
    pthread_t *threads = malloc(thread_num * sizeof(pthread_t));
    thread_arg_t *args = malloc(thread_num * sizeof(thread_arg_t));
    if (!threads || !args) { //检查内存分配是否成功
        perror("malloc");
        return EXIT_FAILURE;
    }

    struct timespec t_start, t_end ;
    clock_gettime(CLOCK_MONOTONIC, &t_start); // 记录开始时间，用于后续计算程序运行
    耗时

    for (int t = 0; t < thread_num; ++t) { //创建并启动每个线程
        args[t].thread_id = t; // 设置线程编号
        args[t].thread_num = thread_num; //设置总线程数
        args[t].N = N; //设置向量长度
        args[t].partial_sum = 0.0; //初始化部分和
        pthread_create(&threads[t], NULL, thread_func, &args[t]);
    }

    double result = 0.0; // 变量 result 存储最终汇总的点积结果
    for (int t = 0; t < thread_num; ++t) { //等待所有线程结束后累加计算结果
        pthread_join(threads[t], NULL);
    }
}

```

```

        result += args[t].partial_sum;
    }
//start 记录结束时间并计算耗时
    clock_gettime(CLOCK_MONOTONIC, &t_end);
    long sec_diff = t_end.tv_sec - t_start.tv_sec;
    long nsec_diff = t_end.tv_nsec - t_start.tv_nsec;
    long total_ns = sec_diff * 1000000000LL + nsec_diff;
    double elapsed_ms = total_ns / 1e6;
//end 记录结束时间并计算耗时

    printf("s=%.2f t=%f(ms)\n", result, elapsed_ms);

    free(threads); //释放分配的资源
    free(args);    //释放分配的资源

    return EXIT_SUCCESS;
}

```